



MODULE E-04

Compact Course Programming. Python - Organisation and Introduction -

Prof. Galyna Tabunshchyk

University of Applied Science and Arts Dortmund

galyna.Tabunshchyk@fh-dortmund.de

Brief Introduction

- Teaching experience
 - Object Oriented Programming, Software Designing and Modelling of IOT Systems, Software and Hardware Quality Engineering, Python for Data Analytics, Embedded Systems Practicum
- Research experience
 - Digital Twins, Applied AI, System Verification, Planning and Monitoring of Software Development Process, Risk Analysis in Software Project Management and Industrial Applications, Reliability of cyber-physical systems

- Publications
 - More then 150 publications.
 - Scopus h-index – 9
 - Researcher Id h-index – 8
 - Author Profiles
 - SCOPUS authorId=56007319800
 - ResearcherID V-1838-2017
 - ORCID orcid.org/0000-0003-1429-5180



- International activities



- Coordinator Erasmus+ KA2 project Work4CE, 2020-2024
<https://work4ce.eu/>
- Coordinator in Ukraine Tempus Project Desire, 2013-2017, Erasmus+ KA 2 Project Bioart, 2017-2020
- Coordinator in NUZP
Erasmus+ KA1 exchange with KU Leuven, FH Dortmund, TU Ilmenau, Carintine University of Applied Science, Transilvania University of Brasov, Politechnica of Madrid, TMMA

Course Targets and Overview

- **Course Target:** Learn basic skills and concepts of computer programming in a various application areas using Python
- **Course Prerequisites:** No programming background required for this course. However, some basic knowledge and experience with computer systems is required.
- **By** the end the student **will be able to**
 - use IDE for developing Python applications
 - use GitHub as a repository for the solutions
 - create simple Python applications
 - implement object-oriented methodology
 - create class hierarchy for solving different practical tasks
 - run Unit tests
 - Create an Minimum viable product (MVP) with OpenAI APIs

Course Concept

- This course is based on QA sessions
 - with a lot of content
 - with small quizzes, tasks and exercises for students during the lecture
 - with student homework solution presentations
- This course requires relevant self-study time (homework assignments, further reading etc.)
- How to be successful in that course
 - Pass the interactive assessments during the QA sessions
 - Do the obligatory homework assignment
 - Develop the project and defend it.

Course Preparation

- Access online learning platform Ilias for compact course material (lectures, homework assignment):
 - https://www.ilias.fh-dortmund.de/ilias/ilias.php?baseClass=ilrepositorygui&ref_id=1690506
- You could use any programming environment during class and for your homework. During QA sessions you will solve small assignments.
 - Python IDLE <https://www.python.org/downloads/>
 - PyCharm <https://www.jetbrains.com/pycharm/>
 - Anaconda <https://www.anaconda.com/products/individual>
 - Jupyterlab <https://jupyterlab.readthedocs.io/en/stable/>
- Install and configure the programming environment
- Create GitHub for your solutions

Course Schedule

Phase I. Preparatory				Phase II. Capstone project				Final project presentation
<i>week 38</i>	<i>week 39</i>	<i>week 40</i>	<i>week 41</i>	<i>week 42</i>	<i>week 43</i>	<i>week 44</i>	<i>week 45</i>	<i>week 46'-47</i>
<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>QA</i>	<i>14./21.11.2026</i>

MODULE E-04 : Compact Course

Programming

Lesson 1

-Python Basics-

Python basics

- Python development platforms
- Python basic types
- Conditions
- Lists, tuples, sets, dictionaries

Python programming language

```
# python
```

```
Python 2.7.4 (default, Sep 26 2013, 03:20:26)
```

```
[GCC 4.7.3] on linux2
```

```
Type "help", "copyright", "credits" or "l  
information.
```

```
>>>
```

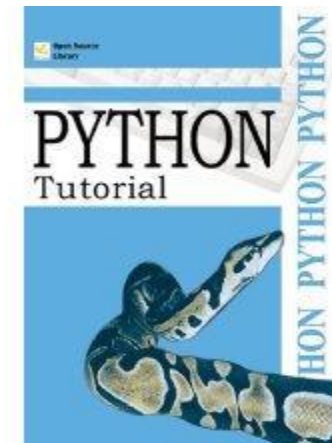
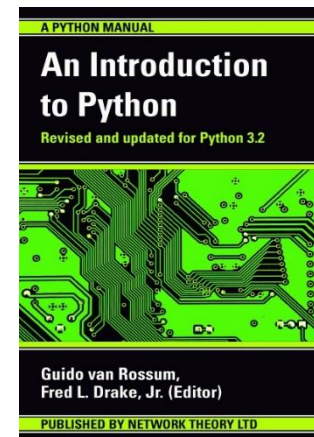


Python

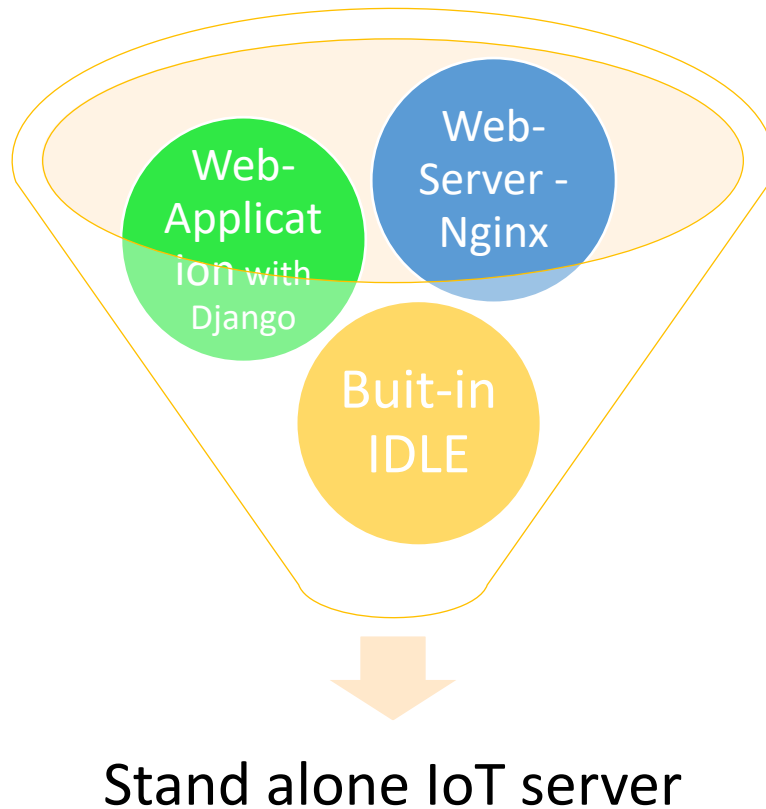
Guido van Rossum



- Python 2.0 was released on October 16, 2000
- Python 3.0, was released on December 3, 2008

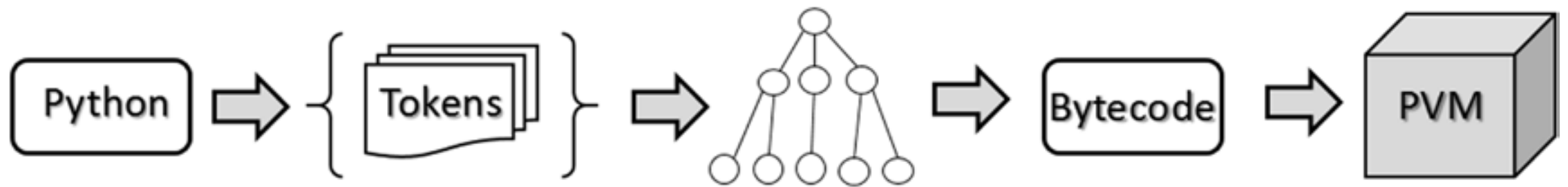


Why Python



- Friendly & Easy to Learn
- Open-source
 - The Python Software Foundation (PSF) is a 501(c)(3) non-profit corporation that holds the intellectual property rights behind the Python programming language
- Python Package Index (PyPI) is a repository of software for the Python programming language:
 - [Web and Internet Development](#)
 - [Database Access](#)
 - [Desktop GUIs](#)
 - [Scientific & Numeric](#)
 - [Education](#)
 - [Network Programming](#)
 - [Software & Game Development](#)

Interpreter



Python Integrated Development Environment (IDLE)

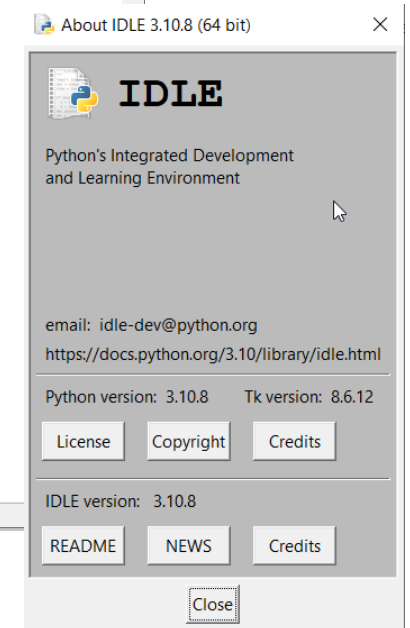
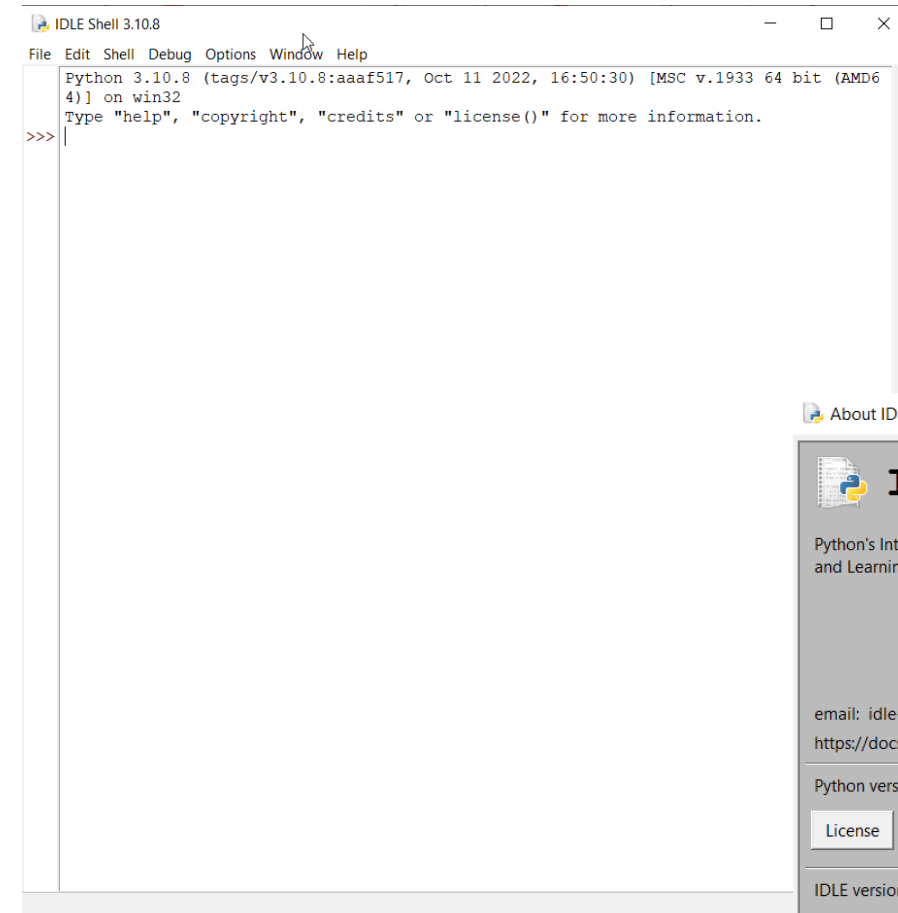


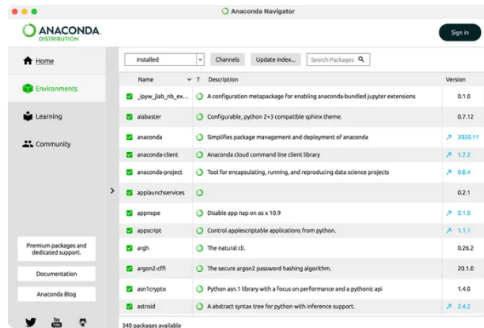
The screenshot shows the Python.org website with the navigation bar and a prominent "Download the latest version for Windows" section. Below this, there are links for other operating systems and a table of active Python releases.

Active Python Releases

For more information visit the Python Developer's Guide.

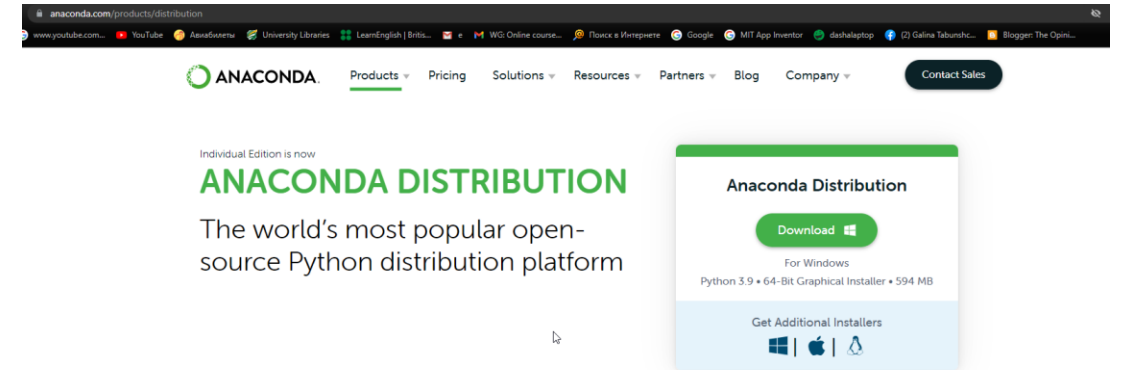
Python version	Maintenance status	First released	End of support	Release schedule
3.10	bugfix	2021-10-04	2026-10	PEP 619
3.9	security	2020-10-05	2025-10	PEP 596
3.8	security	2019-10-14	2024-10	PEP 569
3.7	security	2018-06-27	2023-06-27	PEP 537
2.7	end-of-life	2010-07-03	2020-01-01	PEP 373





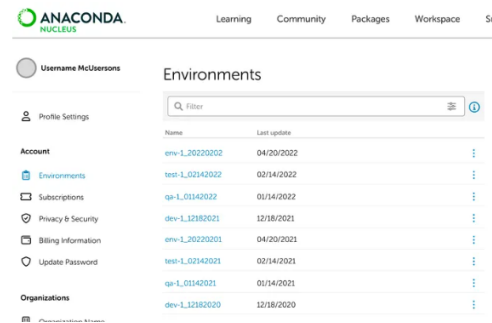
Anaconda Navigator

Our desktop application lets you easily manage integrated applications, packages, and environments without using the command line.



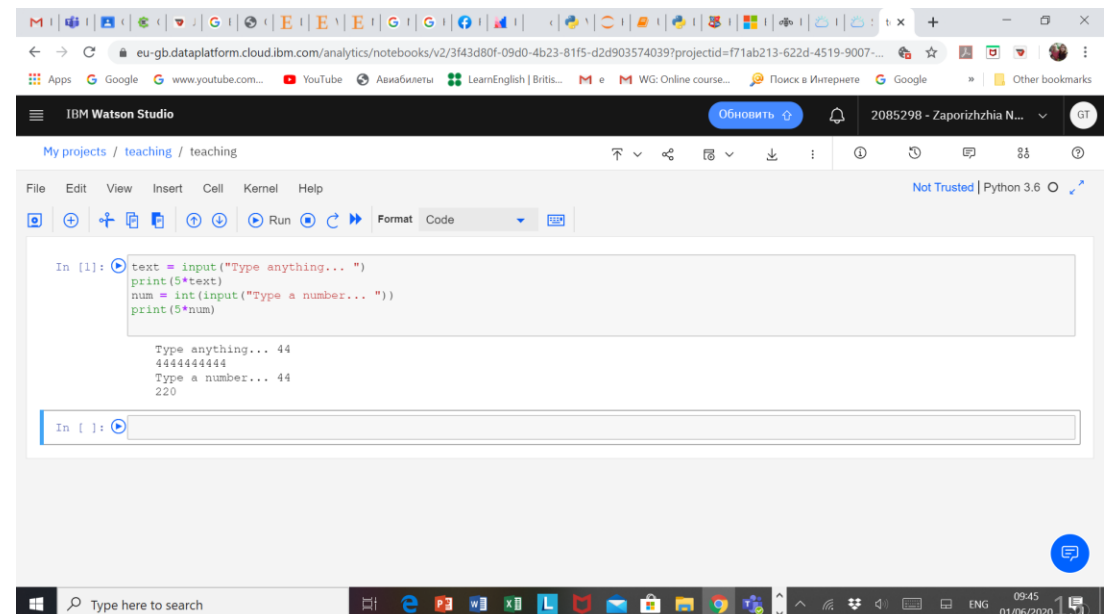
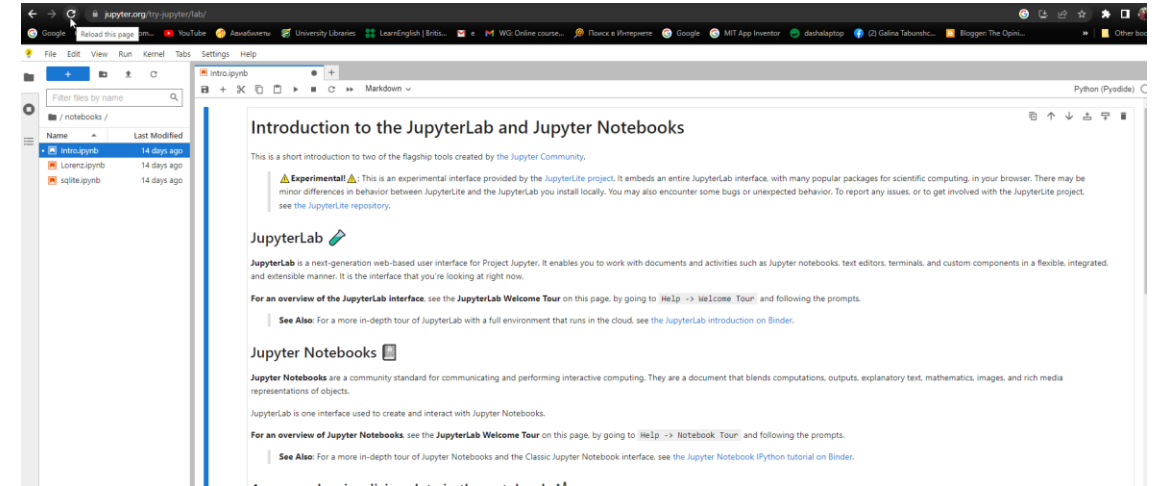
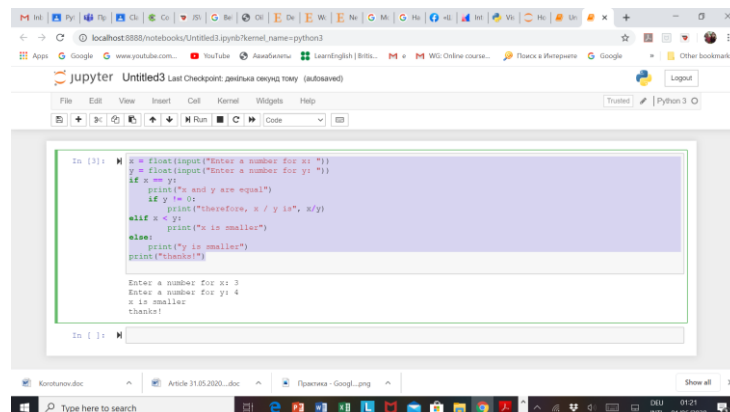
Cloud Environment Backup

Connect Anaconda Navigator to our community portal, [Anaconda Nucleus](#), to securely store your local environments in the cloud.

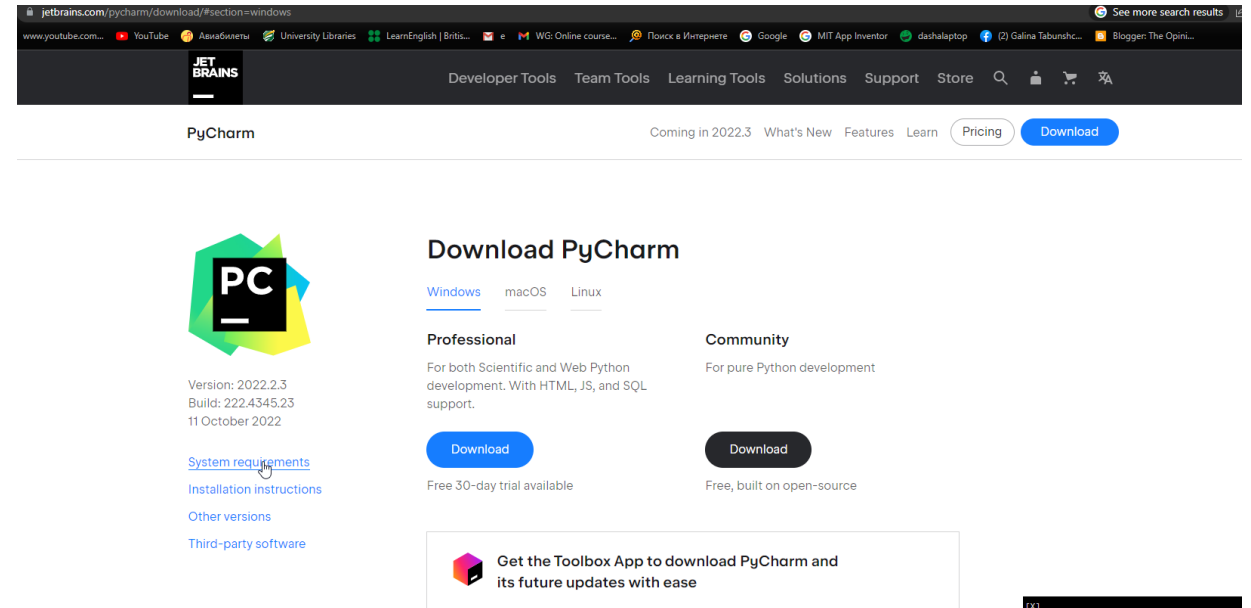
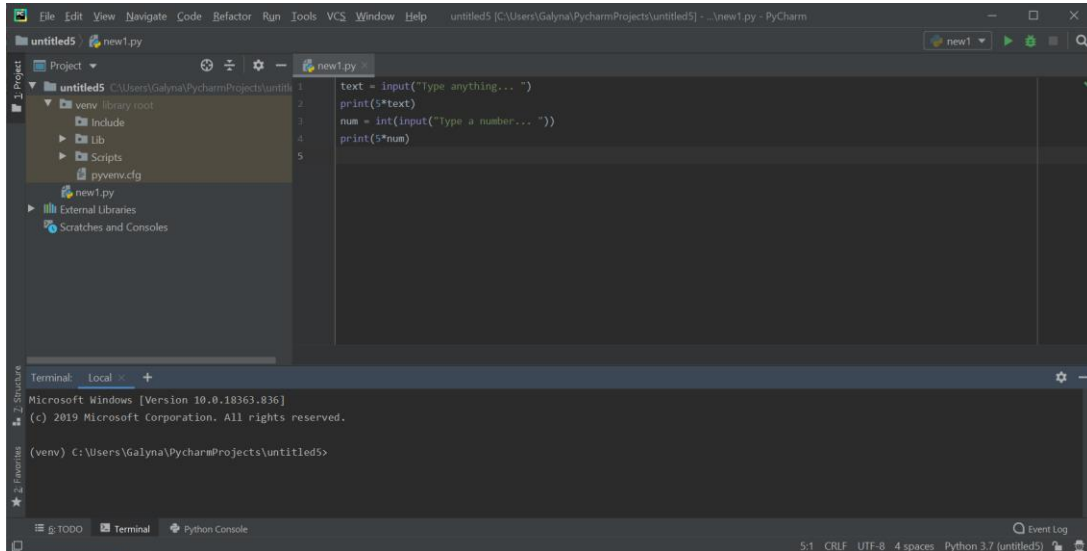


<https://www.anaconda.com/products/individual>

jupyter



PyCharm



[X] Cookies and IP addresses allow us to del web content and to provide you with a pe experience. Our website uses cookies and address for these purposes. [Learn more](#)

Python Operators

- Arithmetic operators
- Logical Operators
- Bitwise Operators
- Relational Operators
- Membership Operators
- Assignment Operators
- Identity Operators

Numeric Types — int, float, complex

Operation	Result
$x + y$	sum of x and y
$x - y$	difference of x and y
$x * y$	product of x and y
x / y	quotient of x and y
$x // y$	floored quotient of x and y
$x \% y$	remainder of x / y
$-x$	x negated
$+x$	x unchanged

WS 2022-2023

Operation	Result
<code>abs(x)</code>	absolute value or magnitude of x
<code>int(x)</code>	x converted to integer
<code>float(x)</code>	x converted to floating point
<code>complex(re, im)</code>	a complex number with real part <i>re</i> , imaginary part <i>im</i> . <i>im</i> defaults to zero.
<code>c.conjugate()</code>	conjugate of the complex number c
<code>divmod(x, y)</code>	the pair $(x // y, x \% y)$
<code>pow(x, y)</code>	x to the power y
$x ** y$	x to the power y

Bitwise Operations on Integer Types

operation	Result
$x \mid y$	bitwise <i>or</i> of x and y
$x \wedge y$	bitwise <i>exclusive or</i> of x and y
$x \& y$	bitwise <i>and</i> of x and y
$x \ll n$	x shifted left by n bits
$x \gg n$	x shifted right by n bits
$\sim x$	the bits of x inverted

Comparison operators

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	$x > y$
<	Less than - True if left operand is less than the right	$x < y$
==	Equal to - True if both operands are equal	$x == y$
!=	Not equal to - True if operands are not equal	$x != y$
>=	Greater than or equal to - True if left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to - True if left operand is less than or equal to the right	$x <= y$

Assignment operators

Operator	Example	Equivalent to	Operator	Example	Equivalent to
=	x = 5	x = 5	**=	x **= 5	x = x ** 5
+=	x += 5	x = x + 5	&=	x &= 5	x = x & 5
-=	x -= 5	x = x - 5	=	x = 5	x = x 5
*=	x *= 5	x = x * 5	^=	x ^= 5	x = x ^ 5
/=	x /= 5	x = x / 5	>>=	x >>= 5	x = x >> 5
%=	x %= 5	x = x % 5	<<=	x <<= 5	x = x << 5
//=	x //= 5	x = x // 5			

String

- Single quotes: 'allows embedded "double" quotes'
- Double quotes: "allows embedded 'single' quotes".
- Triple quoted: '''Three single quotes''', """Three double quotes"""

Python basics

- Variables

- name = "Python example"
- number = 27
- f_n = 1.25

```
print(int(2.3))      2
print(int(-2.8))     -2
print(float(5))      5.0
print(complex('3+5j')) (3+5j)
```

- Output

- x = 1
- print(x)
- x_str = str(x)
- print("my fav num is", x, ".", "x =", x)
- print("my fav num is " + x_str + ". " + "x = " + x_str)

- Input

- text = input("Type anything... ")
- print(5*text)
- num = int(input("Type a number... "))
- print(5*num)

Modules

Import all functions from a module

```
import module_name
```

```
from modname import*
```

Import specific function from module

```
from module_name import  
function_name
```

- from math import pi

Python basics

- if <condition>:
 - <expression>
 - <expression>
- if <condition>:
 - <expression>
 - <expression>
 - ...
- else:
 - <expression>
 - <expression>

- Indentation

```
x = float(input("Enter a number for x: "))
y = float(input("Enter a number for y: "))
if x == y:
    print("x and y are equal")
    if y != 0:
        print("therefore, x / y is", x/y)
elif x < y:
    print("x is smaller")
else:
    print("y is smaller")
print("thanks!")
```

Python basics

- Tuple
 - an ordered sequence of elements, can mix element types
 - cannot change element values, immutable
 - represented with parentheses
 - `te= ()`
 - `t = (2,"mit",3)`
 - `t[0]` - evaluates to 2
 - `(2,"ID",3) + (5,6)` evaluates to `(2,"ID",3,5,6)`
- Lists
 - ordered sequence of information, accessible by index
 - a list is denoted by square brackets, `[]`
 - a list contains elements
 - usually homogeneous (ie, all integers)
 - can contain mixed types (not common)
 - list elements can be changed so a list is mutable

Python list operations

Description	Python Expression	Example	Results
Membership	item in list	3 in [1, 2, 3]	TRUE
Iteration	for x in list:	print x for x in [1, 2, 3]: print(x)	1 2 3
Count from the right	list[-index]	list = [1,2,3]; list[-2]	2
Slicing fetches sections	list[index:]	list = [1,2,3]; list[1:]	[2,3]
Comparing lists	cmp(list_1, list_2)	print(cmp([1,2,3,4],[5,6,7]));	1 -1

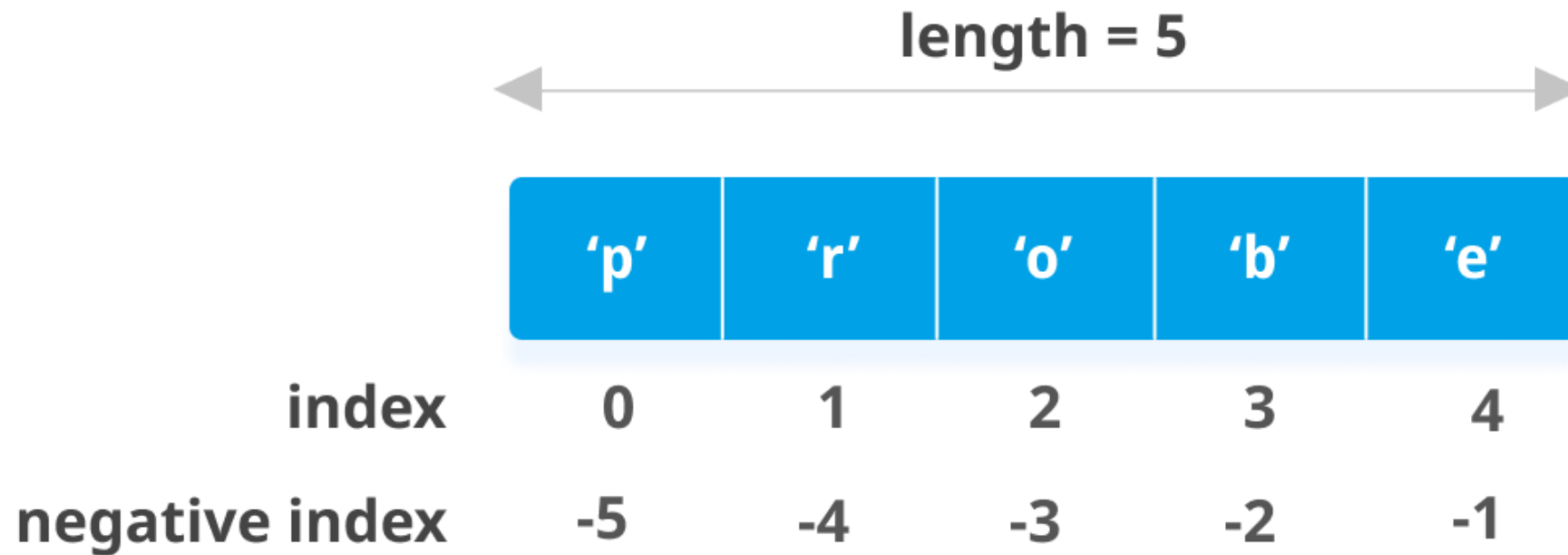
Python list operations

Description	Python Expression	Example	Results
Creating a list of items	[item1, item2, ...]	list = ['a','b','c','d']	['a','b','c','d']
Accessing items in list	list[index]	list = ['a','b','c','d']	list[2] c
Length	len(list)	len([1, 2, 3])	3
Concatenation	list_1 + list_2	[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]
Repetition	list * int	['Hello'] * 3	['Hello', 'Hello', 'Hello']

Python list operations

Description	Python Expression	Example	Results
Return max item	<code>max(list)</code>	<code>max([1,2,3,4,5])</code>	5
Return min item	<code>min(list)</code>	<code>min([1,2,3,4,5])</code>	1
Append object to list	<code>list.append(obj)</code>	<code>[1,2,3,4].append(5)</code>	<code>[1,2,3,4,5]</code>
Count item occurrence	<code>list.count(obj)</code>	<code>[1,1,2,3,4].count(1)</code>	2
Append content of sequence to list	<code>list.extend(seq)</code>	<code>['a', 1].extend(['b', 2])</code>	<code>['a', 1, 'b', 2]</code>
Return the first index position of item	<code>list.index(obj)</code>	<code>['a', 'b', 'c', 1, 2, 3].index('c')</code>	2

Python lists



SLICING

<pre>L=[3,4,5,6,7,8,9,10] print(L)</pre>	<pre>[3, 4, 5, 6, 7, 8, 9, 10]</pre>
<pre>L=[3,4,5,6,7,8,9,10] print(L[:6])</pre>	<pre>[3, 4, 5, 6, 7, 8]</pre>
<pre>L=[3,4,5,6,7,8,9,10] print(L[3:])</pre>	<pre>[6, 7, 8, 9, 10]</pre>
<pre>L=[3,4,5,6,7,8,9,10] print(L[2:7])</pre>	<pre>[5, 6, 7, 8, 9]</pre>
<pre>L=[3,4,5,6,7,8,9,10] print(L[-3:])</pre>	<pre>[8, 9, 10]</pre>
<pre>L=[3,4,5,6,7,8,9,10] print(L[:-3])</pre>	<pre>[3, 4, 5, 6, 7]</pre>

Iterating over a list

- compute the **sum of elements** of a list
- common pattern, iterate over list elements
 - `L=[3,4,5,6,7]`
 - `total = 0`
 - `for i in range(len(L)):`
 - `total += i`
 - `print(total)`
- notice
 - list elements are indexed 0 to `len(L)-1`
 - `range(n)` goes from 0 to `n-1`

```
total = 0
for i in L:
    total += i
print(total)
```

Operations on lists

<pre>L1 = [2,1,3] L2 = [4,5,6] print(L1,L2) L3 = L1 + L2 print(L3) L1.extend([0,6]) L1.append(4) print(L1)</pre>	<pre>[2, 1, 3] [4, 5, 6] [2, 1, 3, 4, 5, 6] [2, 1, 3, 0, 6, 4]</pre>
--	--

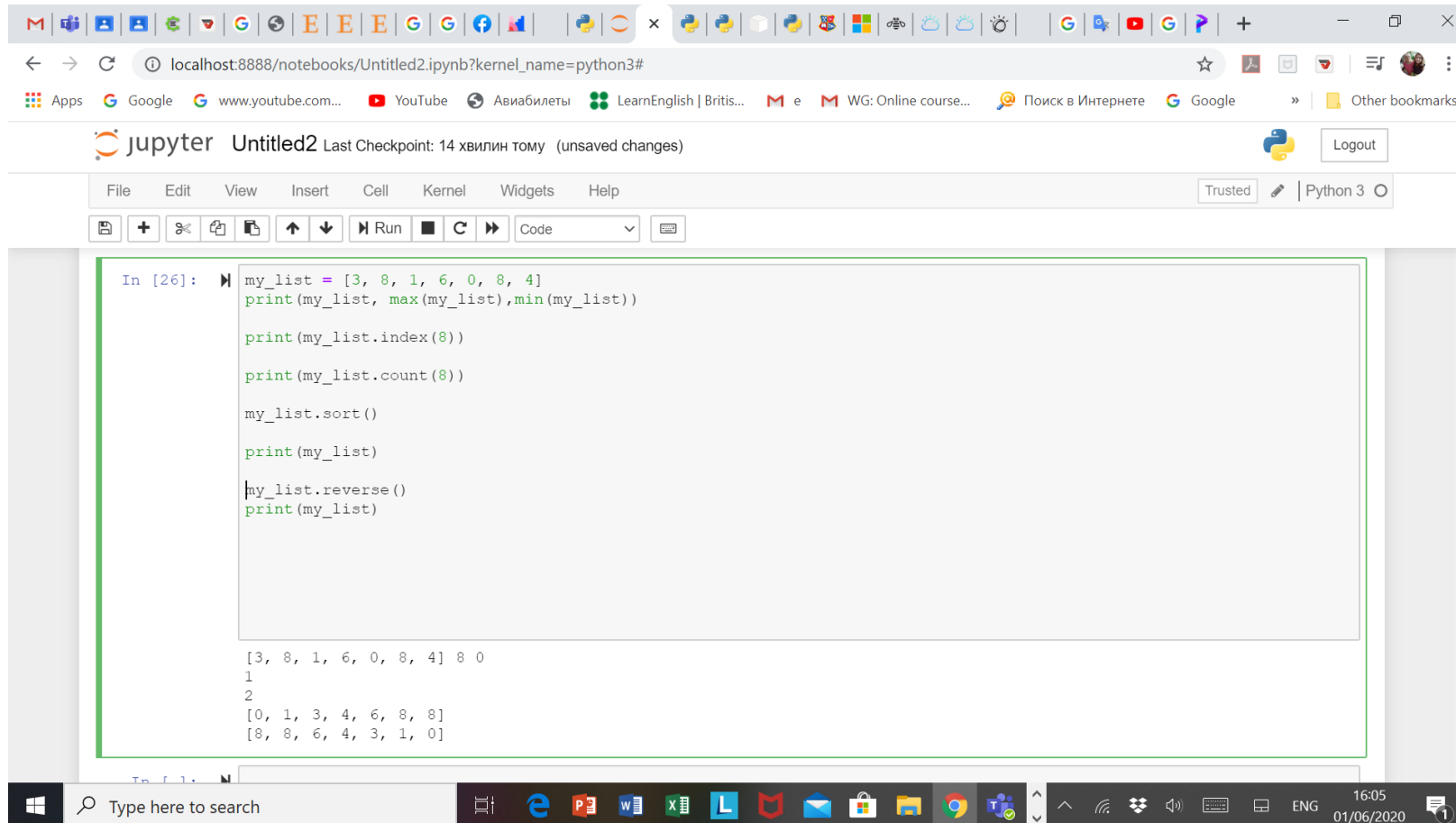
- add elements to end of list with `L.append(element)`
- mutates the list!
 - `L = [2,1,3]`
 - `L.append(5)` # L is now `[2,1,3,5]`
- to combine lists together use concatenation, `+` operator, to give you a new list
- Mutate list with `L.extend(some_list)`

Operations on lists

- delete element at a specific index with `del(L[index])`
- remove element at end of list with `L.pop()`, returns the removed element
- remove a specific element with `L.remove(element)`
 - looks for the element and removes it
 - if element occurs multiple times, removes first occurrence
 - if element not in list, gives an error

<code>L = [2,1,3,6,3,7,0]</code>	
<code>L.remove(2)</code>	
<code>print(L)</code>	<code>[1, 3, 6, 3, 7, 0]</code>
<code>L.remove(3)</code>	
<code>print(L)</code>	<code>[1, 6, 3, 7, 0]</code>
<code>del(L[1])</code>	
<code>print(L)</code>	<code>[1, 3, 7, 0]</code>
<code>print(L.pop())</code>	<code>0</code>
<code>print(L)</code>	<code>[1, 3, 7]</code>

Operations on lists



The screenshot shows a Jupyter Notebook running in a web browser. The browser's address bar displays 'localhost:8888/notebooks/Untitled2.ipynb?kernel_name=python3#'. The notebook's title bar indicates 'Untitled2' with a 'Last Checkpoint: 14 хвилин тому (unsaved changes)' and a 'Logout' button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. The toolbar contains icons for saving, undo, redo, and running code. The code cell, labeled 'In [26]:', contains the following Python code:

```
my_list = [3, 8, 1, 6, 0, 8, 4]
print(my_list, max(my_list), min(my_list))

print(my_list.index(8))

print(my_list.count(8))

my_list.sort()

print(my_list)

my_list.reverse()
print(my_list)
```

The output of the code is displayed below the cell:

```
[3, 8, 1, 6, 0, 8, 4] 8 0
1
2
[0, 1, 3, 4, 6, 8, 8]
[8, 8, 6, 4, 3, 1, 0]
```

The Windows taskbar at the bottom shows the search bar, task view, and several open applications. The system clock indicates the time is 16:05 on 01/06/2020.